```
UP = 38
DOWN = 40
```

38 and 40 are the keycodes for the up and down arrow keys, respectively.

```
paddle =
  up: ->
  down: ->
```

This paddle is for demonstration only; it does nothing.

```
document.onkeydown = (event) ->
  switch event.keyCode
    when UP then paddle.up()
    when DOWN then paddle.down()
```

Assign a handler function to be invoked when a keydown event occurs.

What does this have to do with reading data? Consider this sequence of `keydown` events that occurred during a game of Pong:

```
UP, DOWN, DOWN, DOWN, UP, DOWN, UP, UP, UP
```

It's an array. It's just that at the start of the game you don't have any of the data, and at the end of the game you have all the data. You don't wait until all the `keydown` event data arrives before doing anything—that wouldn't make for a very fun game. You process data as it arrives. The same can apply to any source of data; it can be treated as a source of events.

9.3.2. Data as events

If you treat a data source as a source of events, you can read and process *parts* of it asynchronously as each part arrives. Remember that blocking the event loop causes other events to queue up, as happened when your program spent almost 5 seconds waiting for some data to be loaded and processed:

```
Loading and processing data
Starting server on port 8888
Data loaded in 4.636 seconds
Clock tick at 4.643 seconds
Clock tick at 4.643 seconds
```

What does the data actually look like, though? Compare it to the `keydown` data from the game of Pong: